

LES FILTRES DE RECHERCHE

LA COMMANDE SED



LA COMMANDE SED

- La commande sed est un éditeur ligne non interactif, il lit les lignes d'un fichier une à une (ou provenant de l'entrée standard) leur applique un certain nombre de commandes d'édition et renvoie les lignes résultantes sur la sortie standard.
- Il ne modifie pas le fichier traité, il écrit tout sur la sortie standard.
- sed est une évolution de l'éditeur ed lui même précurseur de vi
- la syntaxe n'est pas très conviviale
- mais il permet de réaliser des commandes complexes sur des gros fichiers.

SED: SYNTAXE

```
sed -e 'programme sed' fichier à traiter
```

Ou

```
sed -f 'programme sed' fichier à traiter
```

- L'option `-n` supprime la sortie standard par défaut, sed va écrire uniquement les lignes concernées par le traitement.
- L'option `-e` n'est pas nécessaire quand on a une seule fonction d'édition.



SED: LA FONCTION DE SUBSTITUTION S

- **s** permet de changer la première ou toutes les occurrences d'une chaîne par une autre.

```
sed "s/nawal/NAWAL/" fichier
```

- change uniquement la première occurrence de la chaîne toto par TOTO

```
sed "s/toto/TOTO/3" fichier
```

- change uniquement la troisième occurrence de la chaîne toto par TOTO

```
sed "s/toto/TOTO/g" fichier
```



- va changer toutes les occurrences de la chaîne toto par TOTO

```
sed "s/toto/TOTO/p" fichier
```

- en cas de remplacement la ligne concernée est affichée sur la sortie standard

```
sed "s/toto/TOTO/w resultat" fichier
```

- en cas de substitution la ligne en entrée est inscrite dans un fichier résultat

```
sed -e "s/[Ff]raise/FRAISE/g" fichier
```

- substitue toutes les chaînes Fraise ou fraise par FRAISE



SED: LA FONCTION DE SUPPRESSION D

- La fonction de suppression **d** supprime les lignes comprises dans un intervalle donné.
- La syntaxe : **sed "20,30d" fichier**

Cette commande va supprimer les lignes 20 du fichier **fichier**.

- On peut utiliser les expressions régulières: **Sed "/toto/d" fichier**

Cette commande supprime les lignes contenant la chaîne toto.

Un interval peut aussi être décrit par deux expressions régulières par une virgule. – **\$sed "/toto/,/toto1/d" Fichier**

Ces deux option d'identification d'un interval sont aussi correcte pour la fonction p.

- Si au contraire on ne veut pas effacer les lignes contenant la chaîne toto (toutes les autres sont supprimées), on tapera:

Sed "/toto/!d" fichier

- En fait les lignes du fichier d'entrée ne sont pas supprimées, elles le sont au niveau de la sortie standard.

LES FONCTIONS P, =

- La commande p (print) affiche toutes les lignes sur la sortie standard.
- Elle invalide l'option -n (affiche les lignes sélectionnées).
- La commande = donne le numéro de la ligne sélectionnée sur la sortie standard.

Ces deux commandes sont utiles pour le débogage, quand on met au point ses programmes sed.

- La commande suivante affiche le numéro de la ligne contenant la chaîne toto.

Sed "/toto/=" fichier



LES FONCTIONS Q, R ET W

- La fonction **q** (quit) va interrompre l'exécution de sed, la ligne en cours de traitement est affichée sur la sortie standard (uniquement si -n n'a pas été utilisée).
- La fonction **r** (read) lit le contenu d'un fichier et écrit le contenu sur la sortie standard.
- La fonction **w** (write) écrit la ligne sélectionnée dans un fichier.

```
sed "/^toto/w resultat" fichier
```

Cette commande va écrire dans le fichier resultat toutes les lignes du fichier fichier commençant par la chaîne toto.

SED: LES FONCTIONS A ET I

- La fonction a (append) va placer un texte après la ligne sélectionnée. La syntaxe est la suivante:

```
a\  
le texte
```

- La fonction i (insert) va placer un texte avant la ligne sélectionnée. La syntaxe est la suivante:

```
i\  
le texte
```

- Si le texte tient sur plusieurs lignes la syntaxe pour le texte est la suivante:

```
ligne 1 du texte\  
ligne 2 du texte \  
ligne n du texte \  
dernière ligne
```

- Exemple :

```
sed 1i\ Debut test1.txt
```

Insert Debut avant la 1 ère ligne du fichier test1.txt

```
sed 4a\ Fin test1.txt
```

Insert Fin après la 4 ème ligne du fichier test1.txt



SED ET LES SOUS CHAÎNES

- La commande:

`Sed -e "s/\([0-9][0-9]*\) /aa\1aa/" fichier`

- La sous expression (sous chaîne) `\([0-9][0-9]*\)` désigne un ou plusieurs chiffres, le premier de la ligne sera entouré des caractères aa.
- La chaîne `to2to` deviendra `toaa2aato`.



EXERCICE

- 1) Transformer les caractères a, b et c du fichier /etc/passwd par un @
- 2) Transformer les caractères a, b et c du fichier /etc/passwd par un @ de la ligne 11 à 20 .
- 3) Transformer les mots du fichier /etc/passwd qui commencent par une voyelles et se terminent par un z par le mot LINUX .



1. `$sed -e 's/[abc]/@/g' /etc/passwd`
2. `$sed -e '11,20 s/[abc]/@/g' /etc/passwd`
3. **`$sed -e 's/^[aeiou].*z$/Linux/g' /etc/passwd`**



LES ALIAS

Il permet de donner un synonyme à une commande ou à un groupement de commandes.

Cela permet de gagner du temps et de simplifier les commandes.

On peut mettre un certain nombre d'alias dans le fichier .profile

La syntaxe est la suivante : `alias nom_alias = 'commande '`

Exemple :

\$ alias l='ls-al | more' # est une commande très #pratique.

\$ alias goappli='cd /home/compte_appli/appli'

Pour obtenir la liste des alias définis : `alias`